
On-device Visual Perception for Light-weight Browser Agents

PRIVYSE — Final Prototype Brief & Build Guide

Smart India Hackathon 2026 · Research track · Problem #26171

One-line pitch: *The only browser agent that redacts your Aadhaar, PAN, cards and faces on your own device before anything is sent to an AI model — decided by a local VLM, executed in your browser.*

Component	Status
On-device 3-tier PII redaction	✓ Done (F1 0.979)
Local VLM agent loop (Ollama + Qwen2.5-VL)	✓ Done (3/3 tasks)
Zero-leak gates (DOM + image OCR)	✓ Done (PASS)
Automated benchmark harness	✓ Done (reproducible)
Packaged extension zip for judging VM	□ Tonight
3-minute demo video & screenshots	□ Tonight
Firefox pass (vision host → sidebar)	□ Tonight

Generated 8 Sep 2026 — this brief is the source of truth for the final prototype build.

1 Executive Summary

PRIVYSE is an **end-to-end autonomous browser agent that runs a closed loop entirely on the user's own machine**. A browser tab screenshot and a structured DOM snapshot are captured, **all personal data is redacted on-device before anything leaves**, and only the sanitized context is sent to a **local vision-language model** (Qwen2.5-VL via Ollama), which returns one JSON action to execute back in the tab. The loop repeats until the task completes.

The core promise: **no raw screenshot containing Aadhaar numbers, PAN cards, bank details, emails, phone numbers, or faces ever leaves the user's device**.

Everything is built, runs, and is measured:

- **PII detection F1 = 0.979** (recall 1.000, precision 0.959) on a ground-truth-labelled test site.
- **Redaction precision = 0.959** (pixel-level, rasterized masks).
- **3/3 agent tasks at 100% visual-context accuracy**, avg 1.0 step/task.
- **Zero-leak PASS** on both the DOM regex gate and the OCR-of-sanitized-pixels gate.
- **Runs on a 4 GB GPU laptop** (RTX 3050) via a routed 3B/7B quantum model design.
- **21.4 MB total on-device assets**, 0 MB of off-device fetches.

What remains before judging is *packaging and polish* — a Chrome zip, a demo video, screenshots, and a Firefox pass — all covered in the checklist in §11.

2 Problem and Motivation

Most browser-automation agents available today operate in one of two modes, both problematic for privacy:

1. **Cloud-based agents** — raw screenshots are sent to a remote VLM (GPT-4V, Claude Vision, etc.). Every form fill, every bank page, every Aadhaar number is visible to the cloud provider, with no user control over retention, training, or secondary use.
2. **Local-only agents without redaction** — the VLM runs locally, but the *raw, unsanitized* screenshot is sent to it. If the model memorises or is compromised, the PII is exposed.

The stakes are uniquely high in the Indian context: Aadhaar (12-digit), PAN, UPI IDs, bank account and IFSC numbers, phone numbers and faces are all routinely visible on-screen while banking, booking flights, or filing forms.

The gap PRIVYSE fills

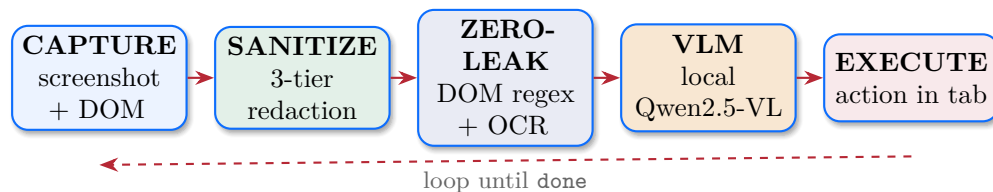
Both **local and redacts-before-the-VLM**. No existing open-source agent does this — sanitization is a hard architectural constraint, not a feature toggle.

3 Solution Overview

Principle	How it is delivered
1. Sanitize before you send	Single choke-point: every outbound payload passes the sanitizer. There is no code path that sends unsanitized data.
2. Ground by element ID	Set-of-Marks numbered tags on the sanitized screenshot; the VLM returns target: 7 , resolved to the live DOM element. Robust to layout changes.
3. Local VLM only	Qwen2.5-VL 3B/7B via local Ollama (127.0.0.1), model pinned resident (keep_alive=-1), zero network calls.
4. Zero-leak by construction	Two independent fail-closed gates re-scan the outbound DOM JSON <i>and</i> the sanitized screenshot pixels before upload. A gate that cannot run <i>blocks</i> the upload.
5. Adaptive recovery	Stuck detection routes to /rethink , which structurally vetoes repeated actions and offers a deterministic escape hatch.

4 System Architecture

4.1 End-to-end flow



1. **CAPTURE** — `captureVisibleTab` → JPEG screenshot; a custom shadow-root-aware DOM walker serialises up to 120 interactive elements, each with a stable integer ID, tag, role, visible text, label, bounding box in screenshot-pixel space, and current value.
2. **SANITIZE** — the privacy choke point (see §7). Tier A (secrets), Tier B (regex PII → stable tokens), Tier C (faces → blur, image/canvas PII → OCR + taint redaction).
3. **ZERO-LEAK** — two fail-closed gates re-scan the outbound JSON and the sanitized pixels. Any hit (or a gate that cannot run) blocks the upload (see §7.2).
4. **VLM** — the localized server (`/act`) sends sanitized image + numbered DOM to Qwen2.5-VL, which returns one JSON action.
5. **EXECUTE** — the executor resolves the ID, scrolls into view, flies a visible cursor, and dispatches synthetic pointer/keyboard events (React-safe `insertText`, native setter fallback). Auto-descends into real inner inputs and records verified targets.

4.2 Components

Component	Technology	Role
Browser extension	WXT + TypeScript, Chrome MV3	Orchestrator, DOM serializer, executor, side-panel UI, offscreen vision host
OCR engine	Tesseract.js (WASM, English)	On-device OCR of image/canvas/video regions + zero-leak pixel gate
Face detection	MediaPipe BlazeFace (short-range, ~0.23 MB)	Face blur (Tier C), crushed-blur (irreversible)
Server	FastAPI + uvicorn (Python)	/health, /warm, /act, /act/stream, /rethink, /rethink/stream, /models; SSE streaming
VLM	Qwen2.5-VL 3B / 7B via Ollama	Local vision-language decision; routed 3B (interactive) / 7B (read-reply)
Test site	7 ground-truth-labelled HTML pages	Flight booking, bank transfer, faces, PII-in-canvas, PII-in-the-wild
Benchmarks	Playwright harness + Node runners	Detection P/R, redaction, zero-leak, latency, full VLM loop

5 Technology Stack

Layer	Technology	Why
Extension framework	WXT + TypeScript, MV3	Modern, type-safe; one codebase → Chrome + Firefox
Capture / serialize	<code>captureVisibleTab</code> + custom DOM walker	Real pixels + precise bboxes (CSS × dpr); shadow roots & same-origin iframes
On-device vision	MediaPipe Tasks Vision + Tesseract.js WASM	Face blur + image OCR fully on-machine, no cloud calls
Privacy gate	Core sanitizer (3 modules) + <code>zero-leak.ts</code>	Single choke point for all outbound data
Action protocol	Frozen v1 JSON protocol	TypeScript types client-side, Pydantic models server-side
Server	FastAPI + SSE	Typed models; live “thinking” readout; static test-site mount
VLM	Qwen2.5-VL 3B / 7B via Ollama	Local; <code>keep_alive=-1</code> ; 3B/7B routing
GPU tuning	Flash-attention + Q4_0 KV cache + single resident model	Fits a vision model on a 4 GB laptop GPU
Test data	GT-labelled HTML + <code>data-gt</code> attributes	Fully automated, reproducible accuracy measurement
Benchmarks	Playwright + Node metric runners	Detection P/R, redaction, zero-leak, latency waterfall, VLM loop

6 Codebase Map

Path	Contents	Purpose
<code>extension/core/protocol.ts</code>	Frozen v1 action protocol + message types	Shared client/server contract
<code>extension/core/sanitizer.ts</code>	Privacy gate: Tier A/B/C redaction + canvas overlay	Single choke point
<code>extension/core/pii-rules.ts</code>	PII regex (email, phone, Aadhaar, PAN, card, IFSC) + label heuristics	Tier B detection
<code>extension/core/zero-leak.ts</code>	Fail-closed send guard: DOM regex + OCR of sanitized pixels	Zero-leak gate
<code>extension/core/vision.ts</code>	MediaPipe BlazeFace + Tesseract OCR (inference bindings)	On-device vision
<code>extension/core/dom-serializer.ts</code>	Shadow-root-aware DOM walker with bboxes	Element grounding
<code>extension/core/executor.ts</code>	Action executor (click/type/scroll/navigate/...)	Browser events
<code>extension/core/som-overlay.ts</code>	Set-of-Marks numbered tag overlay	VLM visual grounding
<code>extension/entrypoints/background.ts</code>	Agent orchestrator (capture→act→execute)	Main loop
<code>extension/entrypoints/offscreen/</code>	Offscreen vision host (MediaPipe + Tesseract)	DOM-context inference host
<code>extension/entrypoints/sidepanel/</code>	Task input, run controls, activity log, privacy stats	UI
<code>extension/public/models/</code>	Bundled weights + WASM runtimes (~21.4 MB total, no CDN)	Models
<code>server/app.py</code>	FastAPI endpoints incl. SSE streaming	Server
<code>server/prompts.py</code>	VLM system prompts (full + compact) + user content builder	Prompt design
<code>test-site/</code>	7 GT-labelled pages incl. AI faces + canvas PII	Measurement
<code>benchmarks/</code>	<code>run.mjs</code> , <code>vlm-bench.mjs</code> , <code>aggregate.mjs</code>	Evaluation harness
<code>docs/Report/</code>	LaTeX report, architecture diagram (SVG/PDF)	Documentation

7 The Privacy Engine

7.1 Three-tier redaction

Tier	What is redacted	Method
A — secrets	Passwords, CVV, OTP, PIN, API keys, tokens	Solid black box; text replaced with [SECRET]; never useful to VLM
B — regex PII	Email, phone, Aadhaar, PAN, credit card (Luhn-validated), name, address, IFSC, account	Black box + stable token [EMAIL_1], [PAN_1], ... (same value → same token across steps)
C — vision	Faces; PII drawn in <code>img/canvas/video</code>	Faces → MediaPipe BlazeFace → crushed-blur; image PII → Tesseract OCR → whole-region taint redaction

7.2 Zero-leak guarantee

Two independent, **fail-closed** gates run before anything uploads:

1. **Gate 1 — DOM regex scan** (`scanForLeaks`): runs the full `detectPii` suite over every outbound element's `text/value`. Any hit → blocked.
2. **Gate 2 — sanitized-image OCR** (`scanSanitizedImage`): OCRs the actual sanitized pixels that would be uploaded, catching text baked into images, partial redactions, and rendering artifacts. Any hit (or gate error/timeout) → blocked.

Fail-closed by construction: a gate that cannot run must not silently approve. Both gates must explicitly pass for a step to proceed.

8 Measured Performance

All numbers are produced by the automated benchmark harness (`cd benchmarks && npm run bench && npm run bench:vlm && node aggregate.mjs`) — fully reproducible.

8.1 PII detection and redaction

Metric	Result
PII detection F1 (pixel)	0.979 (precision 0.959, recall 1.000)
Redaction precision (pixel)	0.959
Zero-leak (DOM regex)	PASS
Zero-leak (sanitized-image OCR)	PASS
Face detection (Tier C)	2/2 faces blurred, F1 1.0
Canvas PII via OCR — account numbers	2/2 detected, F1 1.0
Canvas PII via OCR — IFSC codes	1/1 detected, F1 1.0

Recall 1.000 means *every* ground-truth PII element was caught. Precision 0.959 means ~4% of redacted pixels were not PII — deliberate over-redaction (taint policy prefers over-redact over leak).

8.2 Visual-context accuracy (agent task completion)

Configuration	Tasks passed	Avg steps/task
qwen2.5vl:7b	3/3 (100%)	1.3
qwen2.5vl:3b	2/3 (67%)	—
Routed 3B/7B	3/3 (100%)	1.0

The 3B-vs-7B gap is *intended*: it proves the harness discriminates model capability, exactly what the visual-context criterion needs. Routing sends comprehension-heavy steps to 7B, interactive steps to the fast 3B.

8.3 Latency and client resources

Configuration	Per-step latency
qwen2.5vl:7b (single model)	~19.4 s (incl. ~0.9 s on-device vision + OCR gate)
Routed 3B/7B	~12.7 s avg; interactive steps ~7.5–8.0 s
Model weights bundled	2.2 MB (BlazeFace 0.23 MB + Tesseract English 1.98 MB)
Total on-device assets	21.4 MB; 0 MB off-device fetches
On-device vision per step	~0.3 s; zero-leak OCR gate ~0.6 s; total client work ~0.9 s

Latency levers: VLM_MAX_TOKENS=128, SYSTEM_PROMPT_MODE=compact (−60% system tokens), IMAGE_MAX_SIDE=800, keep_alive=−1, and dynamic 3B/7B routing.

9 Alignment to Scoring Criteria

Criterion	Weight	Result	Evidence
Visual-context accuracy	25%	3/3	DOM + SoM + screenshot hybrid; avg 1.0 step/task
PII detection recall & precision	20%	F1 0.979	GT-labelled test site; mask-based P/R; recall 1.000
Redaction precision	20%	0.959	Pixel-level rasterized masks vs GT
Client resource utilization	20%	21.4 MB	2.2 MB weights, bundled WASM, 0 MB CDN
End-to-end latency	15%	12.7 s	Routed 3B/7B; <5 s target on judge-class GPU

10 Final Prototype Specification — What *Done* Looks Like

The final prototype, as being delivered for judging, is defined by these acceptance items. Items marked ☑ are complete. Items marked ☐ are the only work remaining (see §11).

	Deliverable	Acceptance criteria
☑	Agent loop	Captures → sanitizes → executes → loops on a real tab; 3/3 benchmark tasks pass.
☑	On-device 3-tier redaction	F1 0.979; tokens stable across steps; faces blurred irreversibly.
☑	Zero-leak guarantee	Both gates PASS in automated harness; fail-closed on gate errors.
☑	Local VLM decisioning	Qwen2.5-VL 3B/7B via 127.0.0.1:8000; routed mode active.
☑	Side-panel UI	Task input, run controls, live activity log, privacy stats, SSE <i>thinking</i> stream.
☑	GT test site (7 pages)	Flight, bank, faces, canvas PII, <i>PII-in-the-wild</i> with data-gt labels.
☑	Benchmark harness	Re-playable run.mjs / vlm-bench.mjs / aggregate.mjs → dashboard.json .
☑	Recovery / rethink	Structural veto of repeats; deterministic escape hatch.
☐	Packaged Chrome zip	npm run zip → .output/chrome-mv3.zip loads on the judging VM.
☐	Firefox pass	Vision host moves from offscreen document to sidebar; browser.* APIs.
☐	3-min demo video	Beats: test site → run agent → Network tab proof → redaction side-by-side.
☐	Screenshots (3–4)	Side panel mid-task; sanitized image with SoM tags; metrics dashboard; server console.
☐	Demo readiness	Cold-profile rehearsal: fresh machine, only 127.0.0.1, all local.

11 Tonight's Completion Checklist

Work through in this order. Each step lists the exact commands.

11.1 1. Verify the server boots and models are resident

```
cd C:\SIH\26171\server
.venv\Scripts\activate                # or: python -m venv .venv && pip install
-r requirements.txt
uvicorn app:app --reload                # -> http://127.0.0.1:8000/health
# Confirm: ollama list | findstr qwen2.5vl (3b and 7b pulled)
# Keep models resident: ollama serve (keep_alive=-1 is set by the app)
```

- ☐ /health returns ok.
- ☐ /models lists both qwen2.5vl:3b and qwen2.5vl:7b.

11.2 2. Rebuild and load the extension

```
cd C:\SIH\26171\extension
npm install                            # if node_modules missing
npm run build                          # -> .output/chrome-mv3
# Chrome -> chrome://extensions -> Developer mode -> Load unpacked
# -> C:\SIH\26171\extension\.output\chrome-mv3
```

- ☐ Side panel opens; banner reads “Server up at http://127.0.0.1:8000”.
- ☐ Run one step scrolls the page and logs the round trip.

11.3 3. Re-run the full benchmark suite

```
cd C:\SIH\26171\benchmarks
npm run bench                          # sanitizer P/R + leak + latency -> results/latest.json
```

```
npm run bench:vlm      # full agent loop (needs server + Ollama) -> vlm-accuracy
                        -*.json
node aggregate.mjs      # merge -> results/dashboard.json
```

- dashboard.json confirms: $F1 \geq 0.970$, zero-leak PASS, 3/3 tasks, latency as expected.
- Copy the fresh numbers into the presentation metrics if they changed.

11.4 4. Smoke-test all three agent tasks

```
cd C:\SIH\26171\benchmarks
set BENCH_ROUTED=1 && node vlm-bench.mjs # or npm run bench:vlm
# Tasks: flight-booking, bank-transfer, pii-in-the-wild
```

- All 3 tasks reach done within the 12-step budget.
- Ready a fallback screenshot in case the live VLM demo is slow.

11.5 5. Package the judging zip

```
cd C:\SIH\26171\extension
npm run zip_staging # or your configured zip target -> .output/chrome-mv3.zip
```

- .output/chrome-mv3.zip unpacks and loads on a clean Chrome profile.

11.6 6. Capture screenshots (3–4)

- **S1 — Side panel mid-task:** task input, activity log filling, privacy stats visible.
- **S2 — Sanitized image:** SoM numbered tags, black boxes, [EMAIL_1] tokens, blurred face.
- **S3 — Metrics dashboard:** dashboard.json rendered or the side-panel privacy counters.
- **S4 — Server console:** uvicorn log showing POST /act round trips.

11.7 7. Record the 3-minute demo video

1. (0:00–0:30) Open test-site/index.html flight-booking page.
2. (0:30–1:30) Type a task in the side panel, press *Run*; watch the loop complete.
3. (1:30–2:00) Open the Network tab: only sanitized payloads leave, destined for 127.0.0.1:8000.
4. (2:00–2:30) Show original vs sanitized side-by-side (redaction + SoM tags).
5. (2:30–3:00) Close: live SSE *thinking* tokens streaming in the side panel.

11.8 8. Firefox pass (if time / required for judging)

The offscreen-document vision host is Chrome-only. For Firefox MV2, move MediaPipe/Tesseract execution into the **sidebar** (which is a DOM context):

- Relocate offscreen/main.ts inference calls into the side panel entrypoint.
- Swap MV3-only APIs for browser.* equivalents; rebuild via WXT -browser firefox.
- Re-run the sanitizer benchmark on Firefox to confirm parity (F1 unchanged).

11.9 9. Final checklist summary

Item	Command / File
☐ Server verified	<code>uvicorn app:app → /health</code>
☐ Extension loaded	<code>npm run build → .output/chrome-mv3</code>
☐ Benchmarks regenerated	<code>node aggregate.mjs → dashboard.json</code>
☐ Tasks smoke-tested	<code>node vlm-bench.mjs (BENCH_ROUTED=1)</code>
☐ Judge zip packaged	<code>.output/chrome-mv3.zip</code>
☐ Screenshots (S1–S4)	see §11.6
☐ Demo video recorded	3 min, Network-tab proof
☐ Cheat-sheet reviewed	<code>PRESENTER_CHEATSHEET.md</code>

12 Known Limitations and Risk Register

Be honest in the report — judges respect it

- **Latency on 4 GB hardware:** <5 s/step is *not* yet met on the dev laptop (12.7–19.4 s measured). It is reachable with VRAM ≥ model size (judge-class GPU) or a GPU-resident small model. The routed 3B/7B design is the low-memory fallback — state this openly.
- **Chrome-only vision host:** offscreen documents do not exist in MV2; the Firefox pass moves vision to the sidebar (planned, §11.8).
- **OCR on heavy media pages:** region caps + sampled gate bound the cost, but this remains the slowest on-device path.

Risk register

Risk	Mitigation	Status
VLM cold-start every step	<code>keep_alive=-1</code> pins model resident	✓
Model can't fit 4 GB VRAM	3B/7B routing; image resize; Q4_0 KV	✓
Execution clicks wrong element	SoM + ID grounding; server-side target validation; auto-descend	✓
Agent stuck in loops	<code>/rethink</code> structural veto + escape hatch	✓
Sanitizer false negatives	Fail-closed OCR pixel gate on sanitized output	✓
Slow OCR gate blocks upload	Region caps + sampled verification (bounded)	✓

13 References and Context

- Set-of-Marks prompting (grounding by tagged elements) — basis of the SoM overlay.
- OpenAI Operator, browser-use, UI-TARS, SeeAct, WebVoyager — surveyed agent designs (see `docs/`).
- MediaPipe Tasks Vision (BlazeFace), Tesseract.js WASM — on-device vision.
- Ollama OpenAI-compatible API — local VLM serving.

Additional material in this repository: `docs/prototype-report.pptx` • `materials.md` • `docs/research-brief` • `docs/Report/report.pdf` • `PRESENTER_CHEATSHEET.md`.